

```

1. def operasi_konstan(n):
    operasi = 0 (1 Operasi)

    x = n + 1 (1 Operasi)
    operasi += 1 (1 Operasi)

    print("Hasil:", x) ( 1 Operasi )
    operasi += 1 ( 1 Operasi )

    print("Jumlah operasi:", operasi) ( 1 Operasi )

operasi_konstan(10)

```

Output : Hasil : 11

Jumlah operasi: 2

Kompleksitas waktu $O(1)$: Total nya semua 6 Operasi , Jumlah Langkah tidak bertambah meskipun

Nilai n diperbesar seperti ($n=100$, $n=1000$) . Karena jumlah operasinya tetap konstan, maka kompleksitas waktunya $O(1)$

Kompleksitas ruang $O(n)$: X: Menyimpan nilai $n + 1$. Nilai x bergantung pada nilai n . Semakin besar

n , maka semakin besar juga nilai yang ada pada x , sehingga nilai x bertambah sering bertambahnya n

2. Output : 6

Binary Search : bekerja dengan cara membagi 2 bagian

Kompleksitas waktu $O(\log n)$: $n = 9: 9 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 0$ (4 langkah) / 4 iterasi

Hasil: $T(n) = O(\log n)$

semakin besar data, jumlah langkah bertambah sangat lambat

karena data selalu dibagi dua

Kompleksitas ruang $O(\log n)$: Setiap pemanggilan fungsi butuh ruang tetap untuk menyimpan

variabel low, high, mid, dan parameter lain.

Jumlah pemanggilan rekursif = sama dengan jumlah langkah pencarian = $\log_2 n$.

Jadi, jumlah ruang memori yang dipakai = konstan $\times \log n$ = sebanding dengan $\log n$.

Contoh : Jika $n = 8 \rightarrow \log_2 8 = 3 \rightarrow$ Maksimal ada 3 pemanggilan fungsi sekaligus di memori.

3. Output : Pindah disk 1 dari A ke C

Pindah disk 2 dari A ke B

Pindah disk 1 dari C ke B

Pindah disk 3 dari A ke C

Pindah disk 1 dari B ke A

Pindah disk 2 dari B ke C

Pindah disk 1 dari A ke C

Kompleksitas Waktu : $O(2^n)$

- Persamaan Rekurensi

Jika $n = 1$: hanya ada 1 operasi (cetak perintah), maka $T(1) = 1$.

Jika $n > 1$:

Panggil hanoi($n-1$, ...) $\rightarrow$ butuh waktu $T(n-1)$

1 operasi cetak $\rightarrow$ konstan

Panggil lagi hanoi($n-1$, ...) $\rightarrow$ butuh waktu $T(n-1)$

Persamaannya:

$$T(n) = 2 \times T(n-1) + 1$$

(Angka 1 = operasi cetak, nilainya konstan dan bisa diabaikan saat menghitung notasi besar O)

Setelah menyelesaikan persamaan rekurensi hasilnya seperti ini $T(n) = 2^n - 1$

Nilai konstan -1 tidak berpengaruh apa apa jadi : $O(2^n)$

Kompleksitas ruang : $O(n)$

- Untuk n keping, kedalaman rekursi maksimal = n (pemanggilan berantai: $\text{hanoi}(n) \rightarrow \text{hanoi}(n-1) \rightarrow \dots \rightarrow \text{hanoi}(1)$).

- Setiap pemanggilan butuh ruang tetap untuk variabel n , sumber, bantu, tujuan.

- Total ruang = kedalaman rekursi $\times$ ukuran data per panggilan.

Karena ukuran data nya panggilan tetap, maka kebutuhan memorinya bertambah sebanding dengan nilai n : Jadi $O(n)$